



CLOUD

Summit 2026 臺灣雲端大會

MSSQL Cloud Migration

Leading VLDBs Elegantly Toward Autonomy

從 MSSQL 雲端遷移，看超大型資料庫如何優雅邁向雲端自治

Jerry HSU / 1630 TrackD



Expertise



Problem solving



SQL Server Performance tuning



CDN / WAF optimization



Hybrid cloud solution



Design Thinking coach

ASUSTek infra lead

現為華碩電腦資深基礎架構主管，帶領 SRE 團隊負責華碩電腦官網、AI 雲端服務、全球會員系統、資料中台、ROG 電競官網...等架構規劃、建置與維運。過去三年，專注於解決複雜的基礎架構需求、安全合規、效能議題，確保全球數千萬用戶能夠穩定且高效地存取華碩電腦提供的數位服務。

同時為台灣 SQL PASS 社群志工，對 SQL 效能調教、Azure SQL 雲端架構、資料整合方案有興趣。



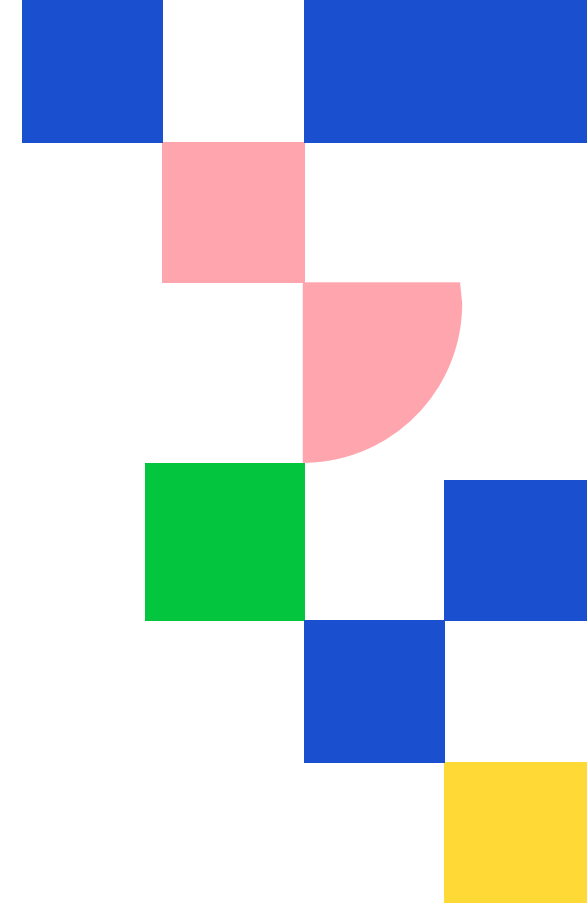
Conference Speaking Highlights

- Building MCP-Driven AIOps: vibe SQL Management 2026 SQLPASS TW
- 讓 SQL Server 聽你說話- MCP 驅動的 AIOps 2025 Hello World Dev Conference – iThome TW
- 從雲原生架構看 AI 助理系統的效能、擴展性與韌性設計 2025 Cloud Summit – iThome TW
- Arc-enabled SQL Server 延伸管理、無限可能 2024 SQL Summit TW
- Think as SQL optimizer, then code for it. 2023 SQLPASS TW
- SQL Server Tuning from the beginning 2022 SQL Summit TW
- Performance tuning from 35% to 4% with Read/Write split & SARGable statement 2021 SQL Saturday Taipei



Agenda

- 大型資料庫雲端遷移的架構決策
- Managed Instance Link 實現地雲近乎零停機遷移
- 雲端資料庫 FinOps 挑戰
- 自動化調校的限制、風險
- 地端 DBA 到雲端數據架構師的思維轉變
- Q & A





**“90% of organizations will adopt
Hybrid Cloud through 2027”**

– Gartner

主流遷移策略 Replatform first, Refactor later

CLOUD MIGRATION 6Rs FRAMEWORK

Six proven strategies to modernize your applications and accelerate cloud value

Rehost

Lift-and-Shift 應用程式環境原封不動移至雲端 IaaS VM 環境

Replatform *

Lift, Tinker, Shift 核心架構不變，部分重構雲端 PaaS 託管型服務

Refactor *

以雲原生架構觀點，重新設計或架構重構



關聯式資料庫通常是最大的支出之一

ENTERPRISE OLTP WORKLOAD – CSP TCO RANKING

Real-world cost distribution for mission-critical OLTP applications (IaaS SQL VM deployment).

資料庫授權費用

vCore / vRAM

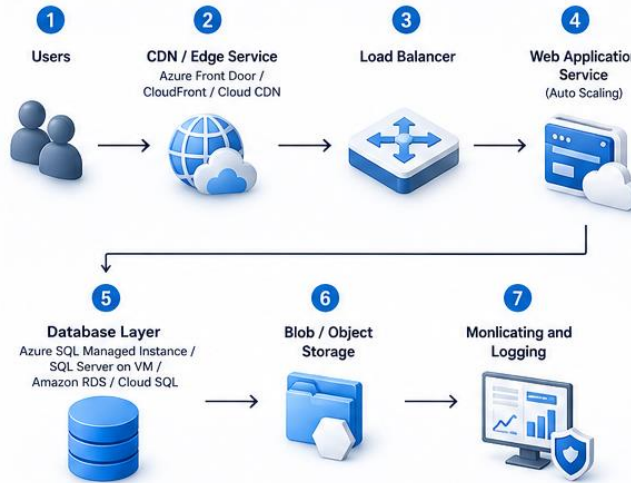
IOPS Premium SSD

HADR / multi Replica

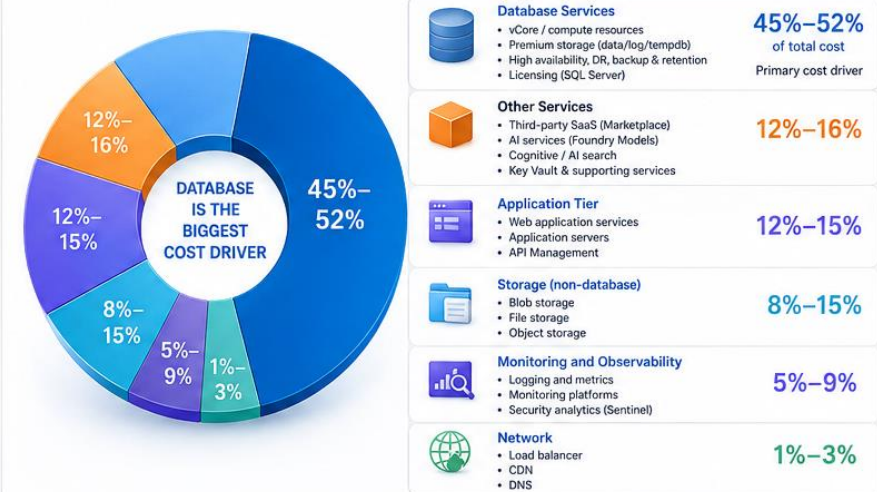


- In enterprise OLTP workloads, the database is the single largest cloud expense.
- Database services typically represent **45%–52%** of total infrastructure cost.

TYPICAL OLTP ARCHITECTURE



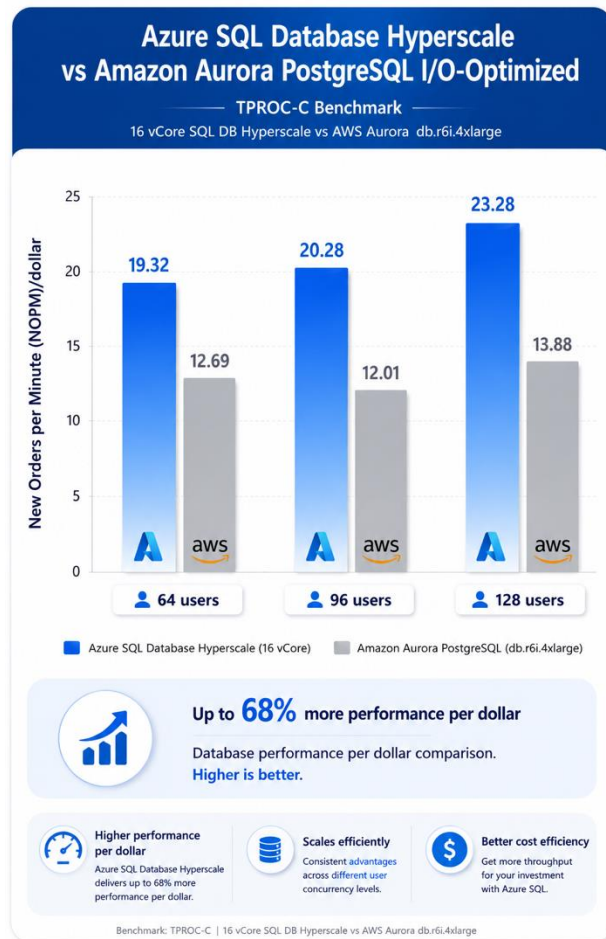
CLOUD TCO BREAKDOWN



TCO RANKING (HIGHEST TO LOWEST)



Focus on database cost optimization for the greatest TCO impact. Right-sizing compute, storage, and high availability design delivers the highest savings.

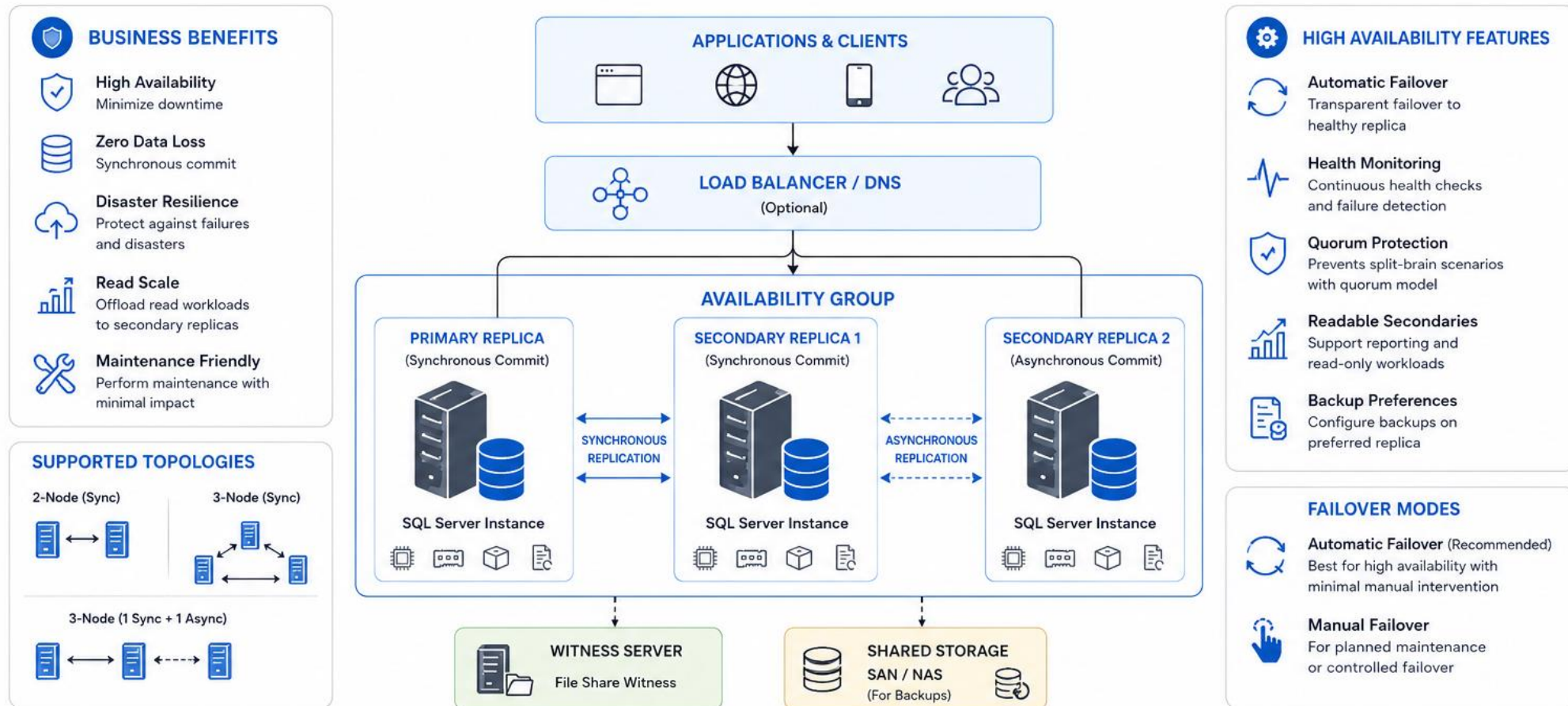


“Cloud Relational Database performance per dollar.”

<https://www.principledtechnologies.com/Azure-SQL-Database-Hyperscale-delivered-better-performance-and-value-at-scale-vs-Amazon-Aurora>

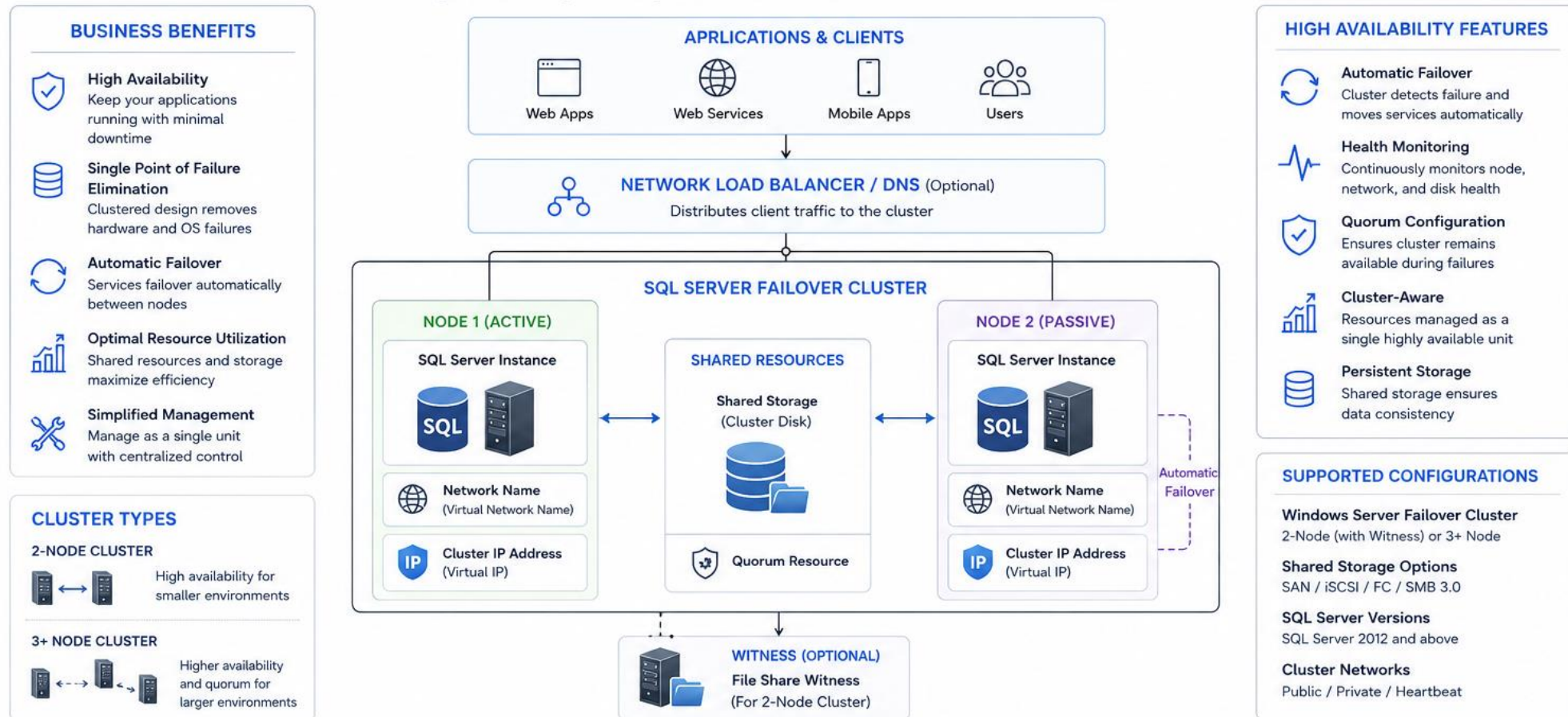
常見的地端 SQL 架構

SQL Server Always On Availability Groups



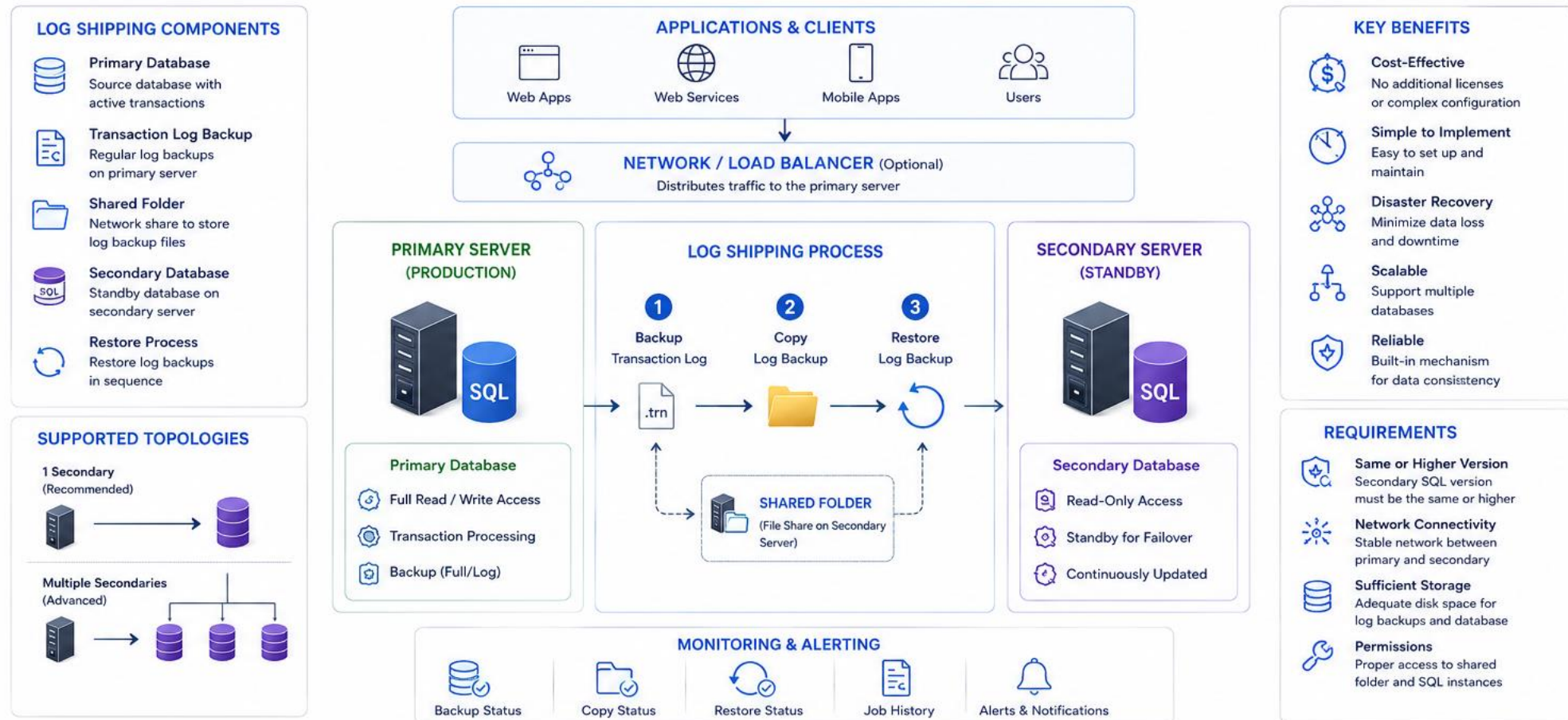
常見的地端 SQL 架構

SQL Server Failover Cluster



常見的地端 SQL 架構

SQL Server Log shipping



雲端 SQL 架構

SQL VM

完整控制權、第三方相容、直接搬移需求

Azure SQL MI

SQL Server 最大相容 (跨庫、SQL Agent)

Azure SQL Database

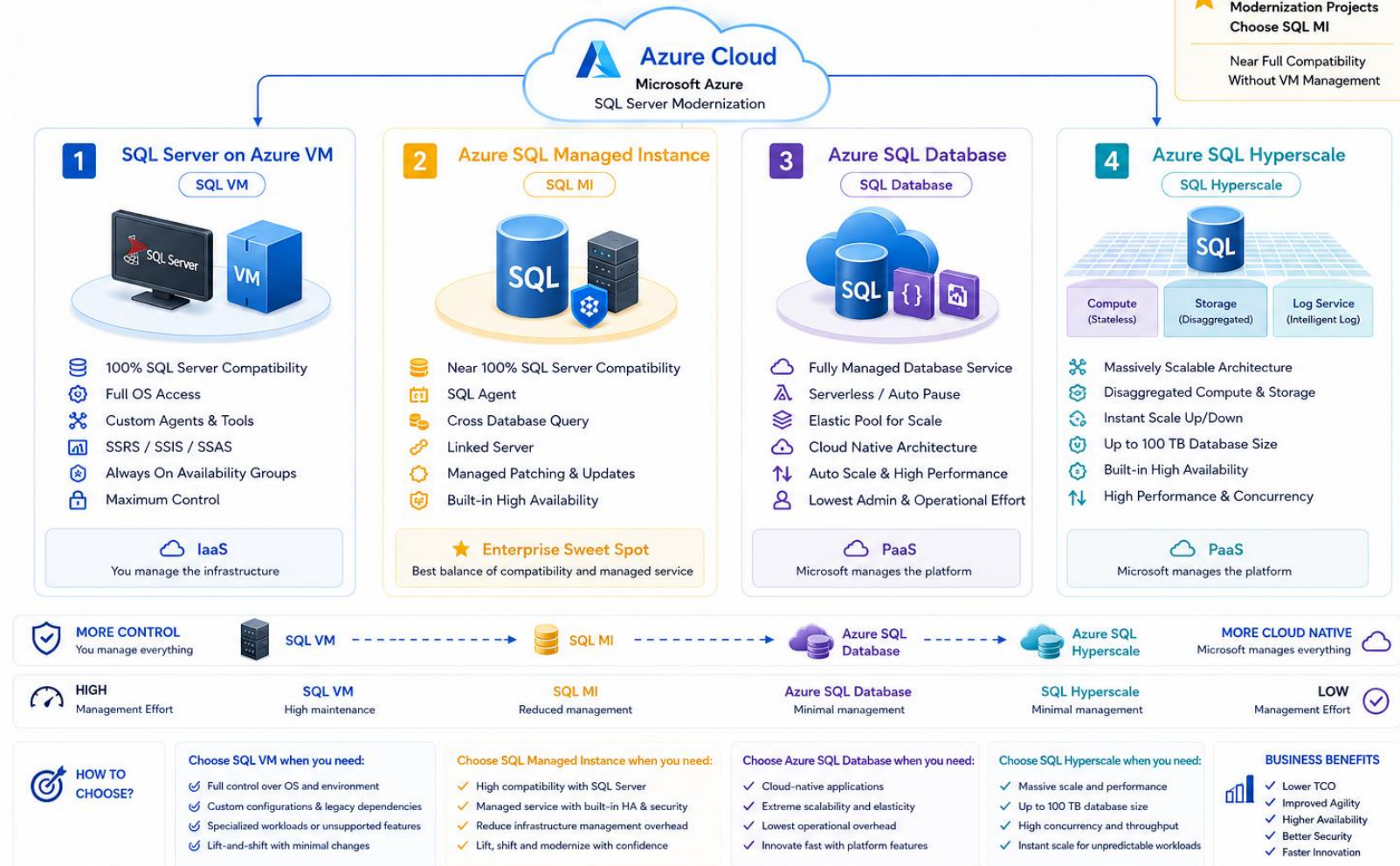
全自動維護、彈性擴展、單一資料庫

Azure SQL Hyperscale

超大資料量、秒級備份、極速擴展應用

AZURE SQL MODERNIZATION OPTIONS

Control vs Compatibility vs Cloud-Native Innovation





**“It's not where you migrate,
but how –
complete, fast, seamless,
and rollback-ready.”**

遷移方案

即時 或 短停機

Distributed AG

沿用既有 AG，failover 切換，停機數分鐘

Managed Instance Link

底層 distributed AG，近即時複寫到 MI

Azure DMS — Online 模式

託管式持續同步，GUI/自動化

Transactional Replication

以複寫達成持續同步的替代做法

遷移方案

離線 或 批次

Native Backup to URL + Restore

備份到 Blob 再還原，最單純

Azure DMS — Offline 模式

一次性搬移，停機到完成

Log Replay Service (LRS) → MI

持續還原 log，手動 cutover

BACPAC / 卸載附加

小型資料庫適用，大型偏慢

三條主流遷移路徑

方案	目標平台	停機時間	複雜度	最適情境
Distributed AG	Azure VM	近乎零 (線上 Failover)	中	已有 AG、要保留完整架構
Managed Instance Link	SQL MI	近乎零 (線上 Failover)	中	上 PaaS 又要最短停機
Azure DMS Online	MI / SQL DB	Cutover 才停機	低-中	託管工具，省去自建複寫遷移

使用 Managed Instance Link 進行遷移

運作原理

底層為 SQL Server Always On 分散式可用性群組 (Distributed Availability Groups) 持續將交易記錄 (Transaction Log) 同步到 SQL Managed Instance

同步期間 MI 資料庫「唯讀狀態」可供驗證測試

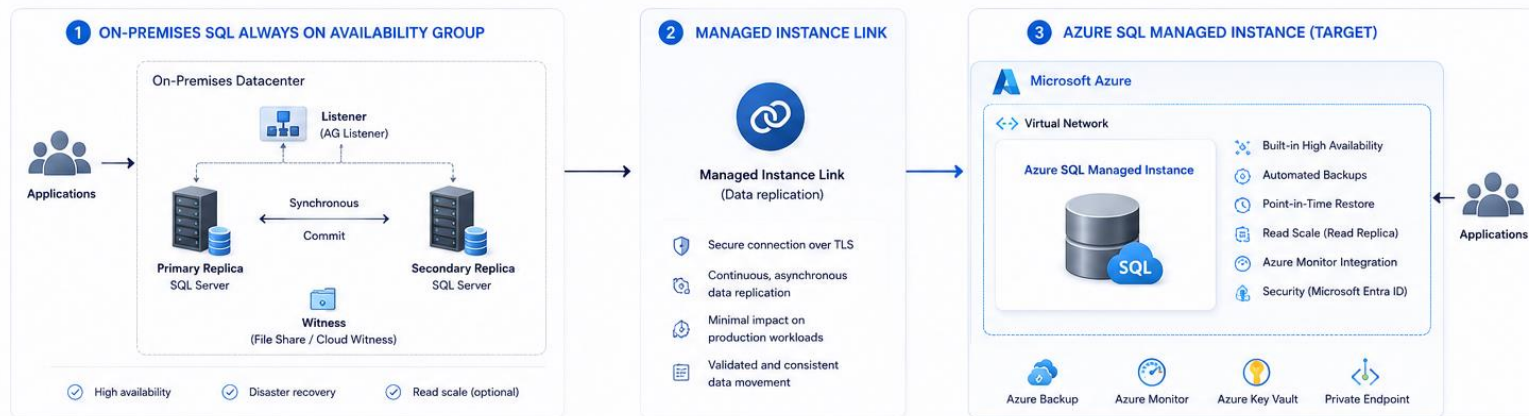
切換透過 Failover 即可完成，為 MI 停機最短的線上遷移路徑

MIGRATING ON-PREMISES SQL AG TO AZURE SQL MI USING MANAGED INSTANCE LINK

Seamless, secure, and low-risk migration with minimal downtime



Managed Instance Link enables fast, secure, and reliable migration of on-premises SQL Server Always On Availability Groups (AG) to Azure SQL Managed Instance with minimal downtime.



MIGRATION PROCESS



KEY BENEFITS

- Minimal Downtime**: Near-zero downtime cutover
- High Reliability**: Built-in HA and automated backups
- Secure & Compliant**: Enterprise-grade security and compliance
- Cost Optimization**: Reduce infrastructure costs and operational overhead
- Future Ready**: Leverage PaaS capabilities and continuous innovation

WHY MANAGED INSTANCE LINK?

- ✓ Fast and efficient migration
- ✓ Low impact on source systems
- ✓ Supports large databases
- ✓ Trusted by Microsoft for SQL Server migrations



使用 Managed Instance Link 進行遷移

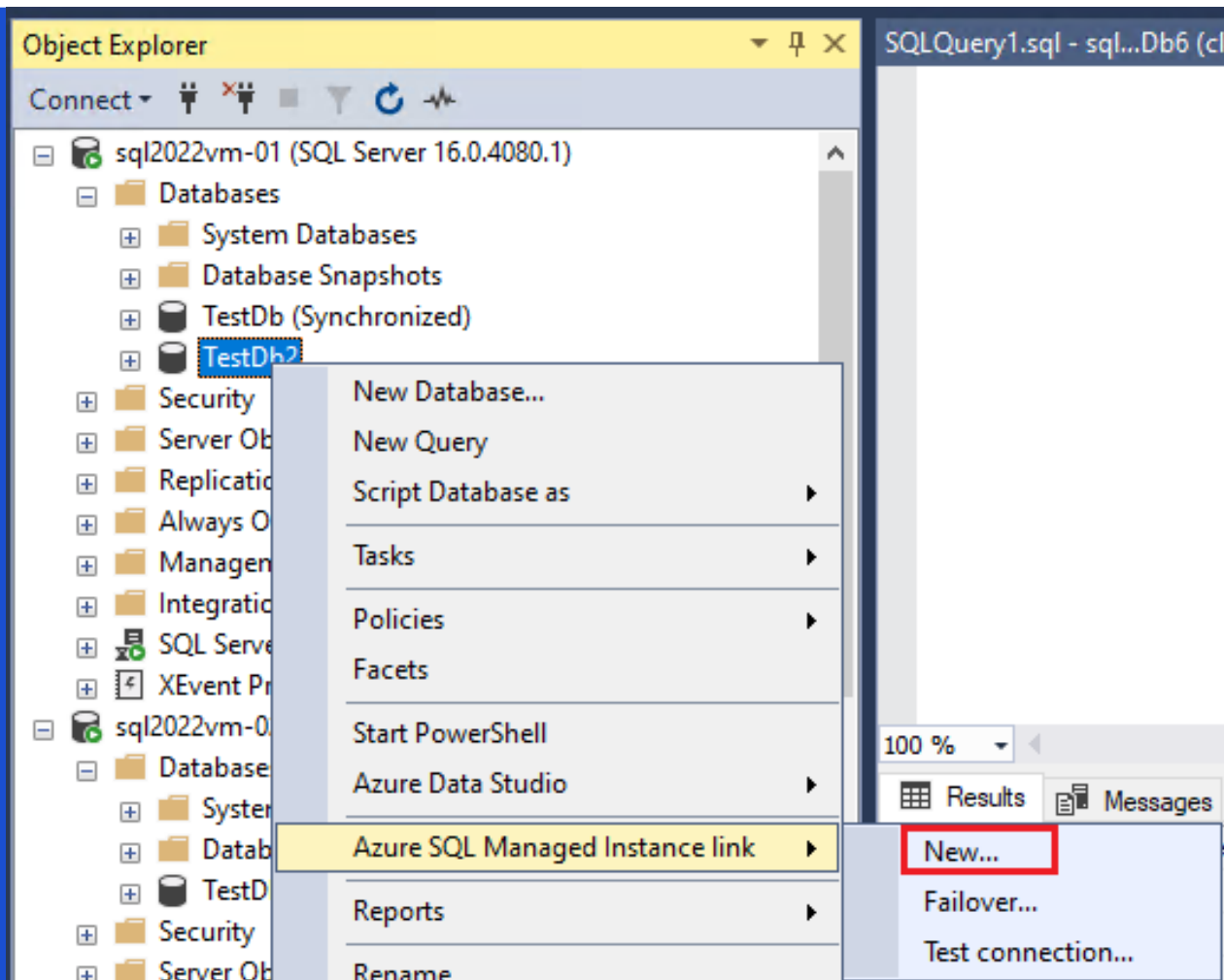
前置條件

支援版本 SQL 2016 / 2017 / 2019 / 2022 / 2025

建立 Azure 與地端的私有網路 (Site-to-Site VPN / ExpressRoute)

開放 Always On Endpoint TCP Port (預設 5022，可自訂)

使用 TDE 的資料庫，需事先完成 TDE Protector (Certificate 或非對稱金鑰) 的移轉與設定



使用 Managed Instance Link 進行遷移

通用限制

- 一個 link 只同步一個資料庫；多 DB 須逐一建立
- 不搬移 instance 層級物件；登入、SQL Agent Job、Linked Server、伺服器組態需另外處理
- 只能往新版相容；轉移後無法退回更舊地端版本 (2022 / 2025 雙向遷移除外)

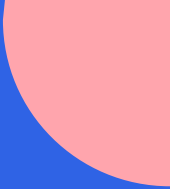
單向遷移 (SQL Server → MI)

SQL Server 2016 / 2017 / 2019

- failover 後無法 fail back
- 視為單程票，切換前需完整驗證

雙向遷移，支持來回切換 (SQL Server ⇌ MI)

SQL Server 2022 / 2025



“While the cloud offers scalability, it also brings complexity. Industry reports indicate that nearly 32% of cloud budgets are wasted due to overprovisioning.”

– A Strategic Guide for Modern Enterprises

DBA 的 KPI 是「不能慢、不能掛」導致「寧可多開、不可出事」

Lift-and-Shift Sizing

照搬地端規格

把地端硬體週期預先配置的規格思維 1:1 帶上雲，資源過度配置直接被搬過來

Risk-Buffered Peak Sizing

為尖峰買保險

DBA 被效能問責、不被成本問責，理性會以 peak 尖峰冗餘進行規劃

Resource Coupling

資源耦合

為了記憶體或 IOPS 需求，需提高 SKU 或 CPU 等級

Limited Elasticity

缺乏彈性

資料庫水平擴展不易，auto scaling 機制有相對情境限制。「沒壞別動」的思維，難以再調動架構

資料庫應用中，以下情境是否似曾相似？

01 Cursor Loop

RD 用 cursor 把資料一列一列地跑迴圈，
本來一個集合運算就能搞定的事，
硬拆為成千上萬次來做。

✓ 原本可以這樣做
集合運算，一次完成

```
UPDATE Orders  
SET Status = 'Done'  
WHERE CreatedDate < '2024-01-01';
```



✗ 卻這樣做了
一列一列跑迴圈 (Cursor)

```
DECLARE cur CURSOR FOR  
SELECT OrderID FROM Orders  
WHERE CreatedDate < '2024-01-01';  
OPEN cur;  
FETCH NEXT...  
UPDATE Orders SET Status='Done'  
WHERE OrderID = @id;  
LOOP ... (數千~數萬次)  
CLOSE cur; DEALLOCATE cur;
```



資料庫應用中，以下情境是否似曾相似？

02 Non-SARGable query

查詢條件在欄位上套函數、隱含型別轉換，
就算建了索引也形同虛設。

✓ 可使用索引 (SARGable)

直接比對欄位

```
SELECT * FROM Users  
WHERE CreatedDate >= '2024-01-01';
```

使用索引，快速定位



✗ 無法使用索引 (Non-SARGable)

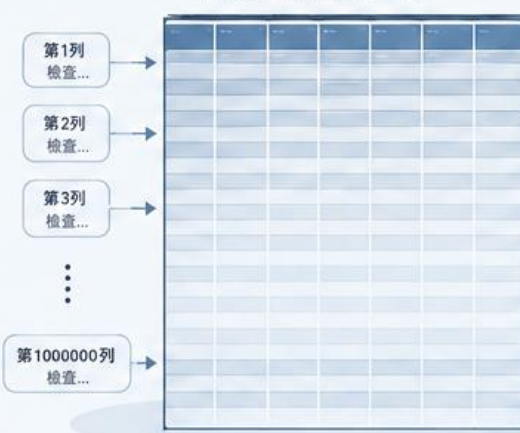
在欄位上套函數 / 隱含轉換

```
SELECT * FROM Users  
WHERE CONVERT(DATE, CreatedDate)  
= '2024-01-01';
```

⚠ 索引形同虛設，只能掃描全表



Users 資料表 (百萬列)



負載過高!



一個本該毫秒完成的查詢，
變成把每一列都翻過一通。
CPU 與 IOPS 只好跟著狂飆。

CPU 使用率

100%

IOPS

狂飆!

執行時間

暴增中...

2 常見 Non-SARGable 寫法

- WHERE YEAR(CreatedDate) = 2024
- WHERE UPPER(Name) = 'ALICE'
- WHERE Col = '123' (欄位為 INT)
- WHERE ISNULL(Status, 0) = 0

建議寫法

- WHERE CreatedDate >= '2024-01-01' AND CreatedDate < '2025-01-01'
- WHERE Name = 'Alice' COLLATE ... (或存成一致大小寫)
- WHERE Col = 123
- WHERE Status = 0 OR Status IS NULL



關鍵重點

避免 Cursor Loop 與 Non-SARGable 查詢，才能讓資料庫發揮索引效能，
降低 CPU、IOPS 與鎖的壓力，讓系統保持在可預期的效能範圍。

從源頭就可能被低估的成本槓桿－效能調校

RI 是折扣 (乘法)，效能調校是降用量 (減法)。先將多消耗的用量消除，折扣才真正划算。

Reserve

財務槓桿

RI / Reserved Capacity / Hybrid Benefit

在既有規格上打折。用量不變，規格鎖定 1-3 年

Performance Tuning

效能槓桿

索引、查詢/執行計畫、統計資訊、減少邏輯讀取 (I/O)

直接降低 CPU / I/O / 記憶體需求，從底層降低 SKU 成本



Cursor 調校案例：效能提升 10.5 倍

透過重構 Cursor 程式為集合式操作 (Set-based)，大幅降低 CPU 負載與回應時間

調校前 (使用 Cursor)

大量逐列處理導致 CPU 長時間高負載、I/O 增加、回應時間延遲

平均 CPU 使用率

48%

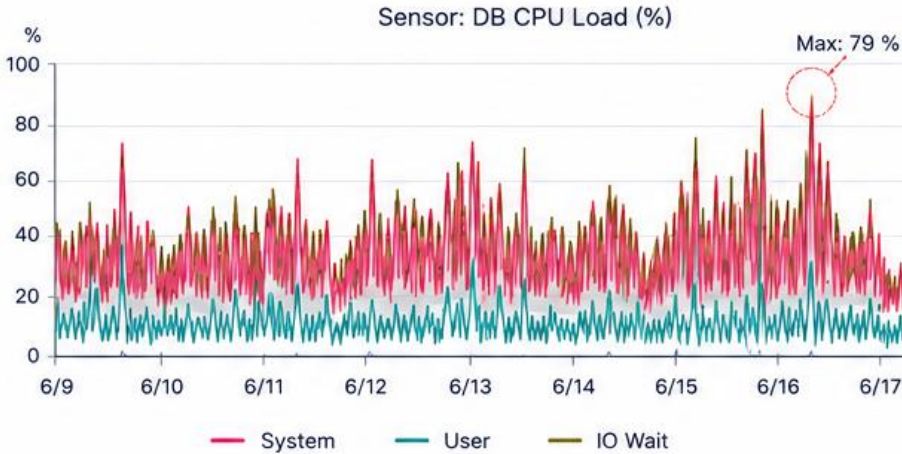
(6/9 ~ 6/17)

CPU 使用率 Peak

79%

資料庫 I/O

高



調校後 (集合式處理)

改採集合式處理，減少迴圈與重複 I/O，CPU 負載顯著下降，整體效能大幅提升

平均 CPU 使用率

12~15%

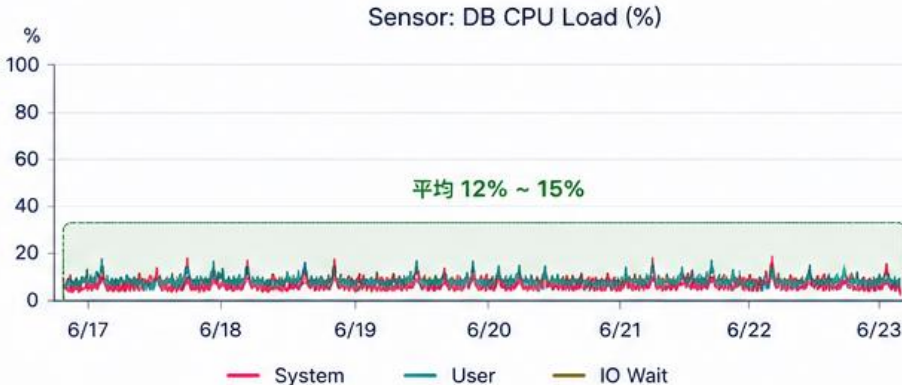
(6/17 ~ 6/23)

CPU 使用率 Peak

22%

資料庫 I/O

低



80th response time

0.56 s

80th response time (優化後)

0.054 s

效能提升

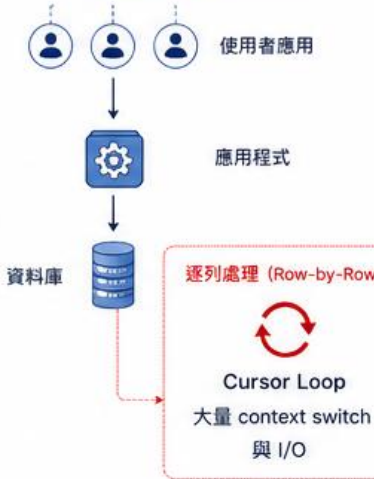
10.5 倍

問題根因：Cursor Loop

```
DECLARE MY_CURSOR CURSOR
FOR
SELECT MagentoId FROM @PrdTmp

OPEN MY_CURSOR
FETCH NEXT FROM MY_CURSOR INTO @Mid
WHILE @@FETCH_STATUS = 0
BEGIN
    INSERT INTO @MagentoIdProds
    SELECT distinct
    @Mid, [Value]
    FROM STRING_SPLIT((
        SELECT [ProductIds]
        FROM @PrdTmp
        WHERE [MagentoId] = @Mid
    ), ',')
    FETCH NEXT FROM MY_CURSOR INTO @Mid
END
CLOSE MY_CURSOR
DEALLOCATE MY_CURSOR
```

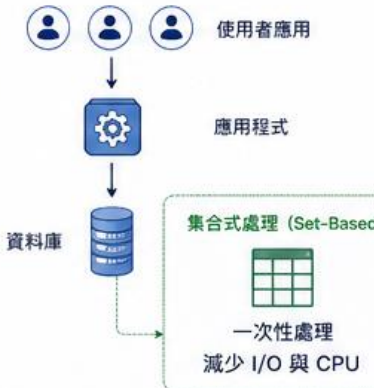
架構與流程 (調教前)



優化作法：Set-Based 改寫

```
INSERT INTO @MagentoIdProds
SELECT p.MagentoId,
       s.[Value]
FROM @PrdTmp p
CROSS APPLY STRING_SPLIT(p.ProductIds, ',') s
GROUP BY p.MagentoId, s.[Value]
```

架構與流程 (調教後)



效能調校 x FinOps 讓每一分雲端支出發揮最大價值

Tune first, commit last

平均 CPU 從 48% 降到 15% → SQL MI 從 32 vCore 降為 16 vCore → 再用 16 vCore 簽 3 年 RI

① 調校前 · PAYG

32 vCore

≈ \$5,900 / 月



Tune

② 調校後 · PAYG

16 vCore

≈ \$2,950 / 月

Tune ↓ 50%



Reserve

③ 16 vCore + 3 年 RI

16 vCore

≈ \$1,970 / 月

Reserve 再 ↓ 33%

若跳過調校，直接把原本 32 vCore 簽 3 年 RI ≈ \$3,940/月；對照先調校降到 16 vCore 再簽 ≈ \$1,970/月
光是「先調校」這一步，每月可再省約 \$1,970，這是 RI 折扣永遠省不到的部分

先調校再綁約 – 雲端資料庫 FinOps 最佳策略

Tune

優化資料庫效能，將 CPU / I/O / 記憶體控制到實際營運所需。這是底層「減法」，並在遷移前就能驗證效果，避免把低效原封不動搬上雲

✓ 降低資源基線

Right-size

以調校後的「資源基線硬體需求」合理評估對應 SKU 規格。保留合理冗餘容量，不再過度配置

✓ 消除 over-provisioning

Reserve

待規格穩定營運，對長期確定的資源簽署 RI / Reserved Capacity / Savings Plan，疊加 Hybrid Benefit。折扣乘在已優化的架構上，效益最大。

✓ 鎖定長期折扣

Automatic Tuning 功能、限制與風險

FORCE_LAST_GOOD_PLAN

預設開啟

偵測「計畫退化」，自動強制返回上一個已知良好的執行計畫

CREATE INDEX

預設關閉

依工作負載自動建立索引，只在低負載時建立並驗證效益

DROP INDEX

預設關閉

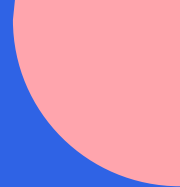
移除 90 天未使用與重複索引；不刪唯一、主鍵索引

限制

- Managed Instance 只支援 FORCE_LAST_GOOD_PLAN
- 不會改動 t-sql / schema / non-SARGable — 核心調教仍仰賴 DBA

風險

- 自動建索引增加寫入/儲存負擔,可能與既定索引策略衝突
- DROP INDEX 可能誤刪罕用索引(如月底報表)
- FORCE_LAST_GOOD_PLAN 對特定參數值未必最佳

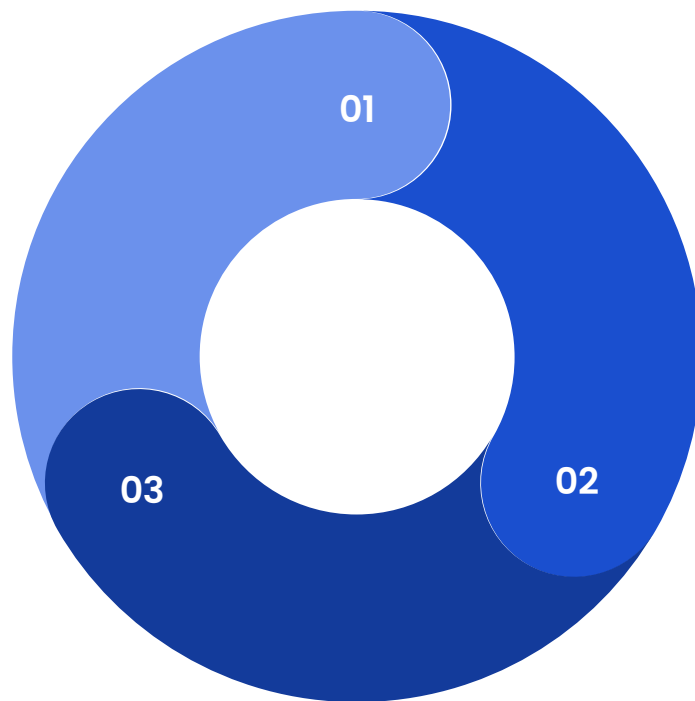


Next Generation DBA

DBA 思維轉變

FinOps by Tuning

將效能轉換為成本



Invite AI to the table

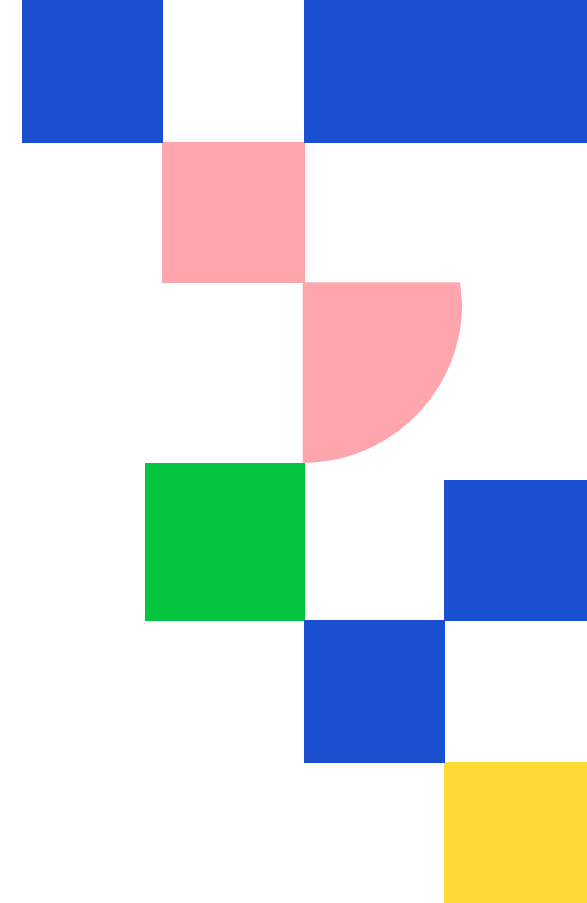
釋放 AI Ops 邊界

Intelligence by Data

這個世界需要更多的資料

Key Takeaways

- 雲端成本最大的浪費來自於過度配置 (Overprovisioning)
- 遷移策略與路徑, 往往比搬去哪裡重要的多
- 資料庫通常是最昂貴且最容易被過度配置的雲端資源之一
- 持續效能優化 Tuning 是 FinOps 最重要的底層邏輯
- RI 只能讓你『便宜地浪費』; 唯有 Tuning 讓你『不再浪費』



Thank You

